

AVANCE DE PROYECTO

Programación Orientada a Objetos II

Pruebas en Postman

Listar, crear, buscar, actualizar y eliminar recursos.



Asignatura	Programación Orientada a Objetos II
Plataforma	IntelliJ IDEA, Postman
Nombre	Arelys Brigitte Obando Castillo
Fecha	01 de junio de 2026

Avance Backend: Entidad Player

Objetivo

Implementar la primera entidad del backend conectada a PostgreSQL para validar el flujo completo entre controlador, servicio, repositorio y base de datos.

Archivos creados

- **Player.java**
 - Entidad JPA que representa a un jugador dentro del sistema NightBite. Esta clase se relaciona con la tabla players en PostgreSQL.
- **PlayerRepository.java**
 - Repositorio encargado de comunicar la entidad Player con la base de datos mediante Spring Data JPA.
- **PlayerService.java**
 - Servicio que contiene la lógica principal para listar, buscar, guardar, actualizar y eliminar jugadores.
- **PlayerController.java**
 - Controlador REST que expone los endpoints HTTP para consumir la información de jugadores desde Postman o desde la aplicación Android.

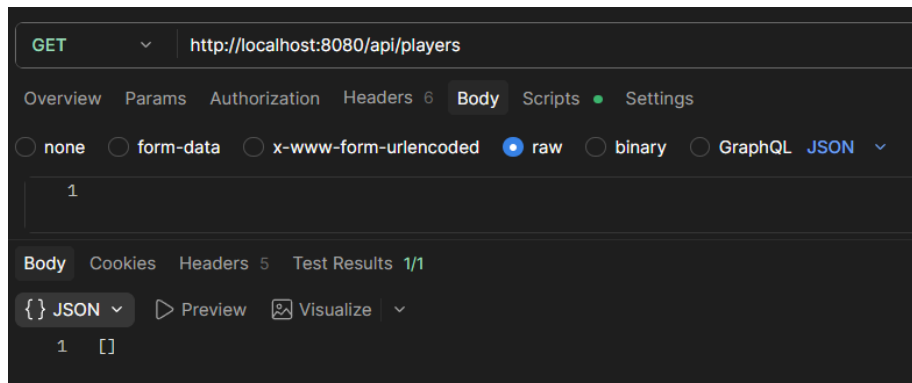
Endpoints probados

Método	Endpoint	Descripción
GET	/api/players	Lista todos los jugadores registrados.
GET	/api/players/{id}	Busca un jugador por su identificador.
POST	/api/players	Registra un nuevo jugador.
PUT	/api/players/{id}	Actualiza los datos de un jugador existente.
DELETE	/api/players/{id}	Elimina un jugador por su identificador.

Evidencias: Las pruebas fueron realizadas en Postman y verificadas en PostgreSQL.

1. Listar jugadores

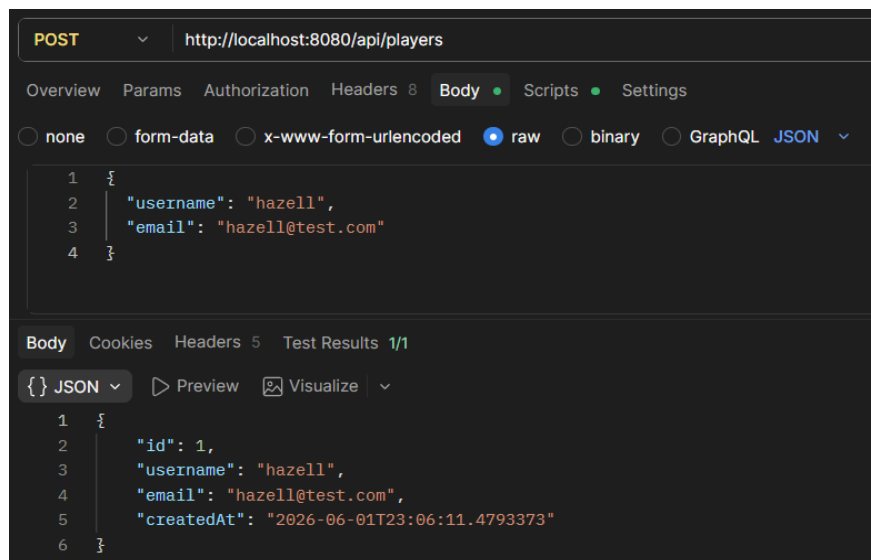
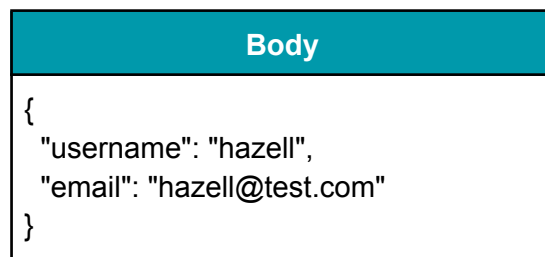
GET <http://localhost:8080/api/players>



- El endpoint funciona, pero todavía no hay datos.

2.1. Crear jugador

POST <http://localhost:8080/api/players>

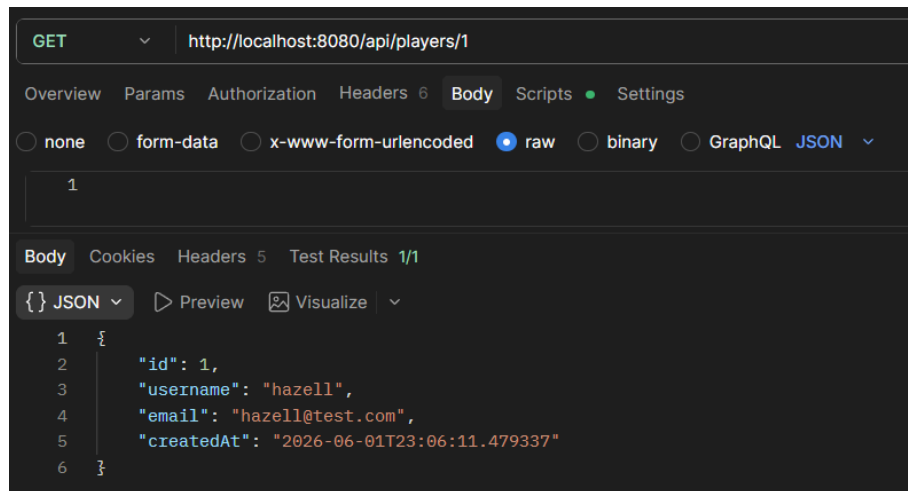


2.2. Verificar en PostgreSQL

	id_player [PK] bigint	created_at timestamp without time zone (6)	email character varying (100)	username character varying (50)
1	1	2026-06-01 23:06:11.479337	hazell@test.com	hazell

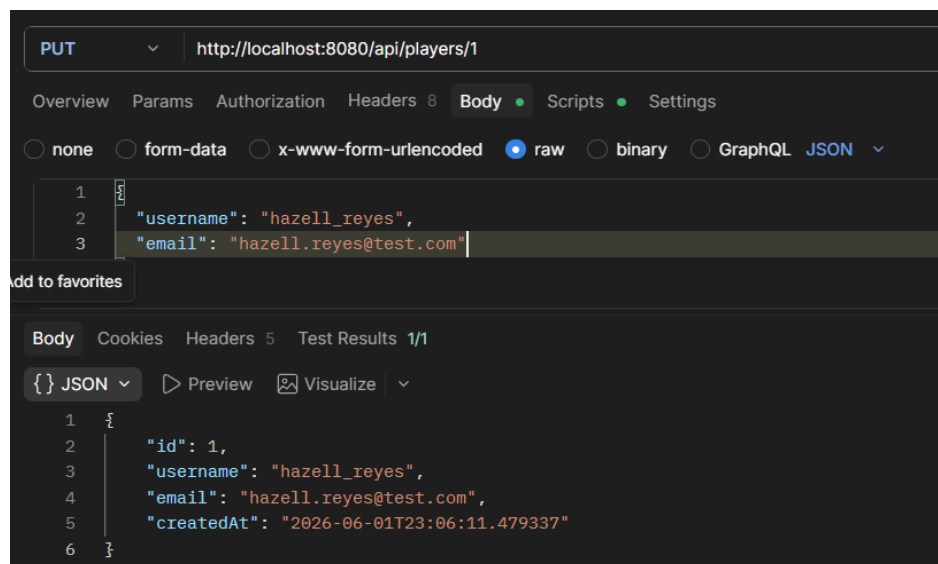
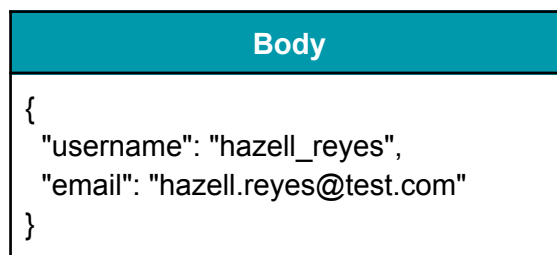
3. Buscar jugador por ID

GET <http://localhost:8080/api/players/1>



4.1. Actualizar jugador

PUT <http://localhost:8080/api/players/1>

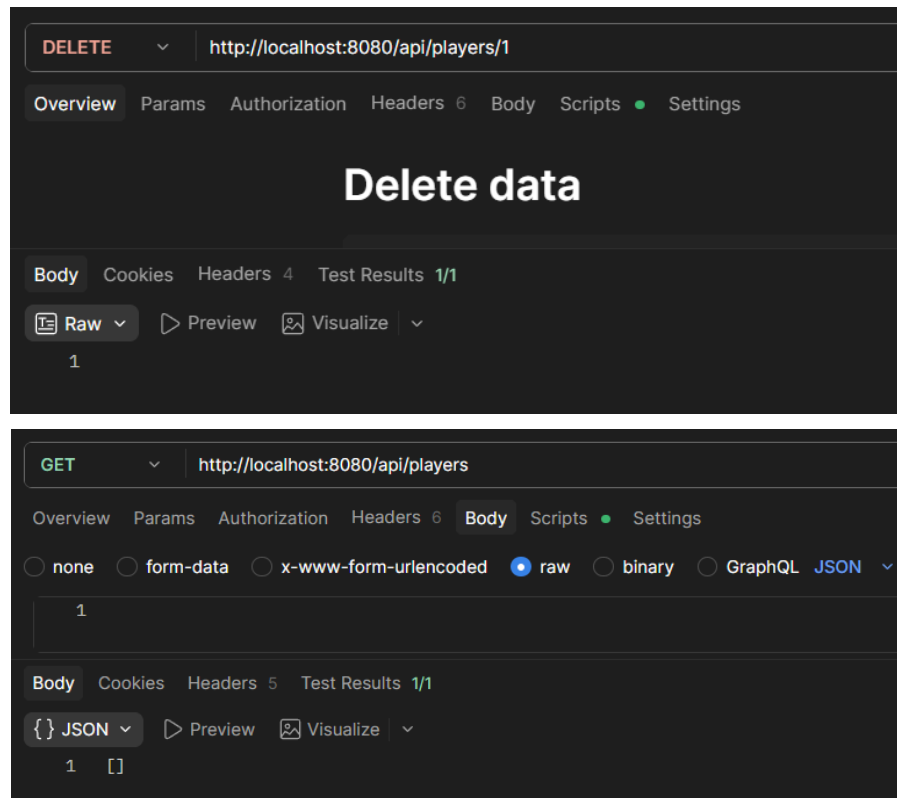


4.2. Verificar en PostgreSQL

	id_player [PK] bigint	created_at timestamp without time zone (6)	email character varying (100)	username character varying (50)
1	1	2026-06-01 23:06:11.479337	hazell.reyes@test.com	hazell_reyes

5.1. Eliminar jugador

DELETE <http://localhost:8080/api/players/1>



5.2. Verificar en PostgreSQL

<code>id_player</code> [PK] bigint	<code>created_at</code> timestamp without time zone (6)	<code>email</code> character varying (100)	<code>username</code> character varying (50)

Resultado

Se validó correctamente la conexión entre Spring Boot y PostgreSQL mediante una entidad inicial. Esto confirma que el backend puede recibir peticiones HTTP, procesarlas y almacenar datos en la base de datos.